Formação Completa

AWS Developer Associate



Domínio 2: Segurança na AWS

Bem-vindos ao domínio de segurança na AWS. Aqui você aprenderá a proteger aplicações e dados de forma robusta.

Autenticação e Autorização

Introdução em Segurança: Conceitos

Autenticação

Processo que verifica **quem você é**. Confirma a identidade do usuário antes de permitir acesso aos recursos.

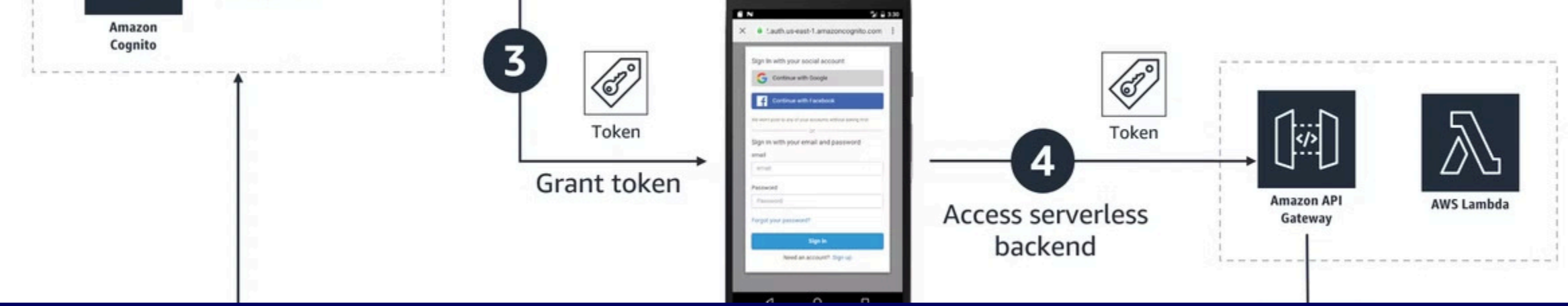
Autorização

Define **o que você pode fazer**. Determina quais permissões e recursos um usuário autenticado pode acessar.

Diferença Fundamental

Autenticação ocorre antes da autorização. Você precisa provar quem é antes de receber permissões específicas.

Na AWS, o IAM gerencia ambos os processos, garantindo que apenas usuários autenticados acessem recursos específicos conforme suas permissões definidas.



Autenticação e Acesso Federado

Verificação de Identidade

Confirma quem é o usuário ou entidade que tenta acessar recursos.

Acesso Externo

Permite que usuários de domínios externos acessem recursos AWS.

Gerenciamento Centralizado

Simplifica o login e integra sistemas legados com a AWS.

Como funciona o Acesso Federado?





Benefícios do Acesso Federado

Gerenciamento Centralizado

Simplifica a administração de identidades em um único local.

Login Simplificado

Usuários acessam múltiplos sistemas com uma única credencial.

Integração Flexível

Conecta sistemas legados com infraestrutura moderna na AWS.

Desafios do Acesso Federado



Configurações Precisas

Exige ajustes técnicos detalhados para funcionar corretamente.



Políticas Robustas

Necessita de regras de segurança bem definidas.



Prevenção de Acessos

Requer controles para evitar autorizações indevidas.

Exemplo de Implementação em Python com PyJWT

```python
import jwt
import datetime

# Chave secreta para assinatura (em prática, use uma chave segura)
SECRET_KEY = 'minha_chave_secreta'

def emitir_token(usuario):
    payload = {
        'user': usuario,
        'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
    }
    token = jwt.encode(payload, SECRET_KEY, algorithm='HS256')
    return token

def validar_token(token):
    try:
        decoded = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
        return decoded
    except jwt.ExpiredSignatureError:
        return "Token expirado"
    except jwt.InvalidTokenError:
        return "Token inválido"

# Exemplo de uso
token = emitir_token("usuario_exemplo")
print("Token emitido:", token)
print("Token validado:", validar_token(token))
```

Autenticação e Acesso Federado

Prós	Contras
Gerenciamento centralizado	Configurações precisas necessárias
Login simplificado	Políticas de segurança robustas
Integração segura	Risco de acessos indevidos

Tokens

O que são

Credenciais digitais que comprovam identidade e permissões de um usuário.

Permitem comunicação stateless entre cliente e servidor.

Como funcionam

Após autenticação, um token é gerado e incluído no header de cada requisição.

Elimina necessidade de gerenciar sessões no servidor.

Exemplo

O cliente (por exemplo, um aplicativo web ou mobile) envia credenciais para o servidor de autenticação:

```
POST /login HTTP/1.1
Host: api.exemplo.com
Content-Type: application/json

{
    "username": "usuario123",
    "password": "senhaSegura"
}
```

Se as credenciais estiverem corretas, o servidor retorna um **token JWT (JSON Web Token)**:

```
{
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

JWT (JSON Web Tokens)

JSON Web Tokens para transmissão segura de informações entre partes.

Estrutura do JWT

Tokens JWT são compostos por três partes separadas por pontos:

- **Header:** Contém o tipo de token e algoritmo de assinatura
- **Payload:** Armazena as claims (informações) do usuário
- **Signature:** Garante a integridade e autenticidade do token

Características principais

- Compacto e autocontido
- Pode ser criptografado
- Transmitido via HTTP headers, URL ou corpo da requisição
- Stateless - não precisa consultar banco de dados
- Amplamente utilizado em APIs RESTful

Aplicações na AWS

Na AWS, os JWTs são frequentemente utilizados em conjunto com Amazon Cognito para autenticação e autorização de usuários em aplicações web e móveis. Também podem ser implementados em APIs Gateway para controle de acesso e em soluções de Single Sign-On (SSO).

OAuth2 e OIDC: Autenticação e Autorização

Protocolos padrão para autenticação e autorização, focados na emissão e validação de tokens.

OAuth2

- **Foco: Autorização** (o que o usuário pode fazer?)
- Protocolo de autorização para acesso delegado a recursos.
- Permite que aplicações acessem recursos em nome do usuário.
- Lida principalmente com *Access Tokens* (para acessar recursos).

OIDC (OpenID Connect)

- **Foco: Autenticação** (quem é o usuário?)
- Camada de identidade sobre OAuth2.
- Permite verificação segura da identidade do usuário.
- Adiciona o *ID Token* (contém informações sobre a identidade do usuário autenticado).

Fluxo OAuth2

1

Solicitação

Aplicação solicita autorização ao usuário.

Autorização

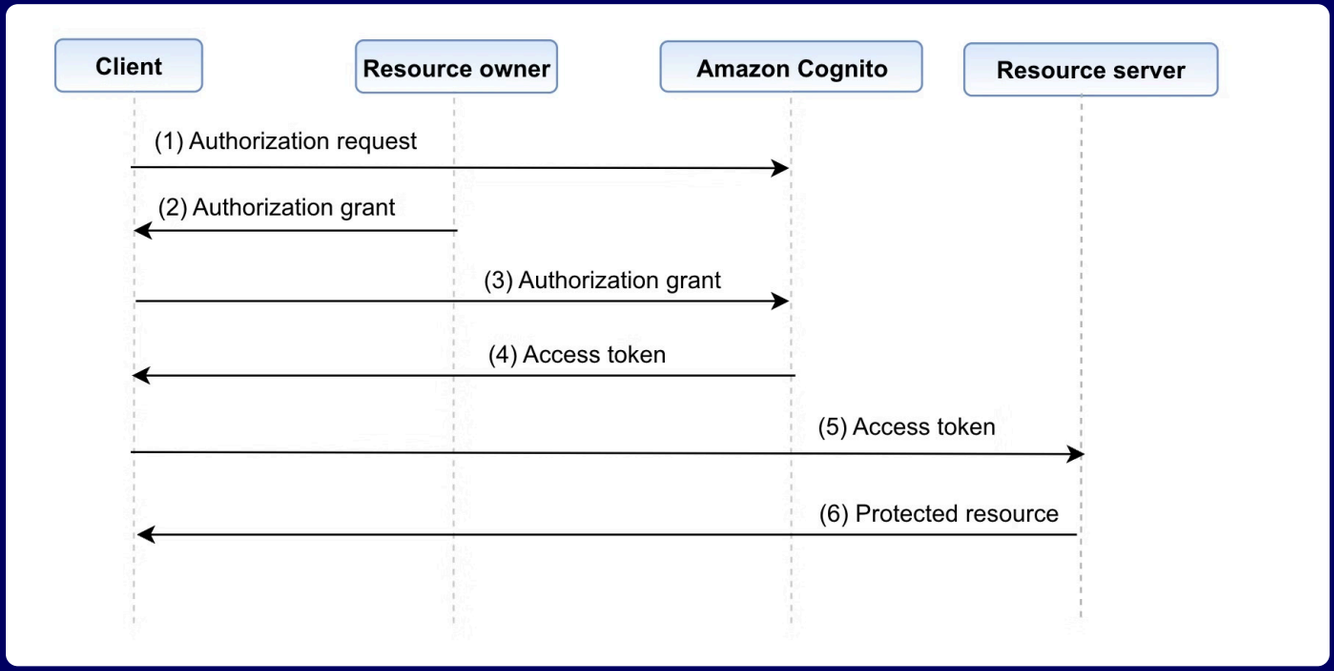
Usuário concede permissão para acesso específico.

Token

Servidor emite token de acesso para a aplicação.

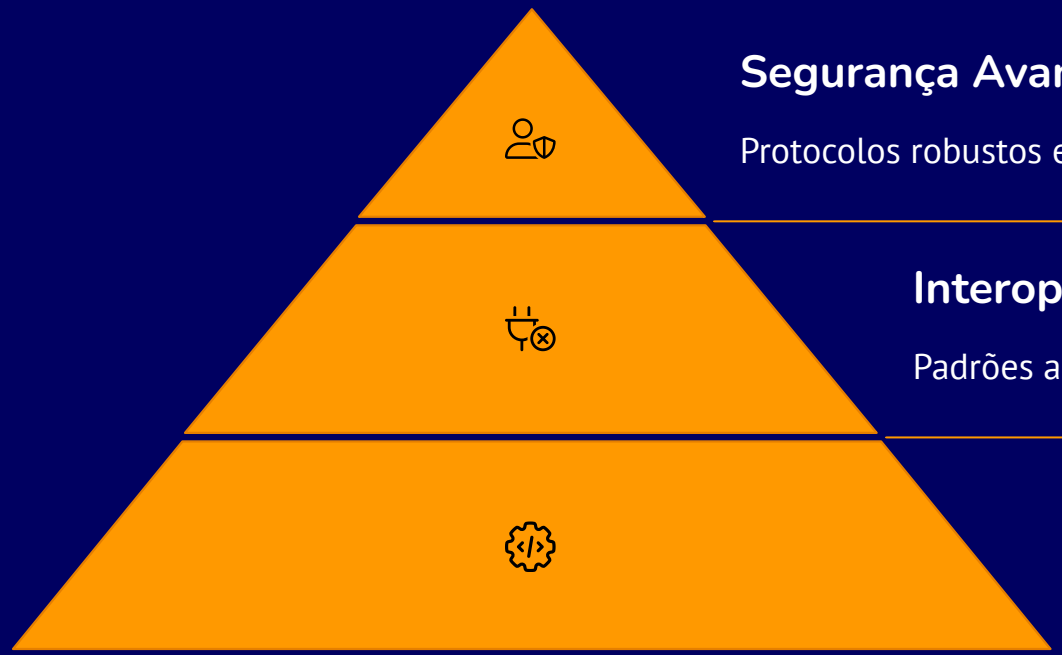
Acesso

Aplicação usa token para acessar recursos protegidos.



Leia mais em: https://aws.amazon.com/pt/blogs/security/how-to-use-oauth-2-0-in-amazon-cognito-learn-about-the-different-oauth-2-0-grants/

Benefícios do OAuth2/OIDC



Segurança Avançada

Protocolos robustos e amplamente testados.

Interoperabilidade

Padrões adotados por grandes provedores de identidade.

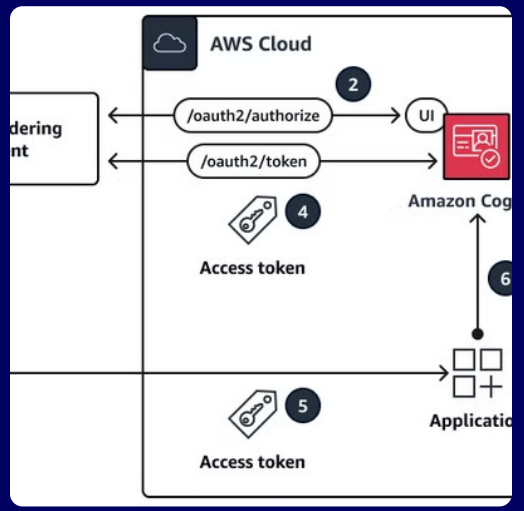
Flexibilidade

Suporta diversos fluxos de autenticação.

Casos de Uso









Diferentes cenários de aplicação dos mecanismos de autenticação e acesso na AWS, desde empresas até dispositivos IoT.

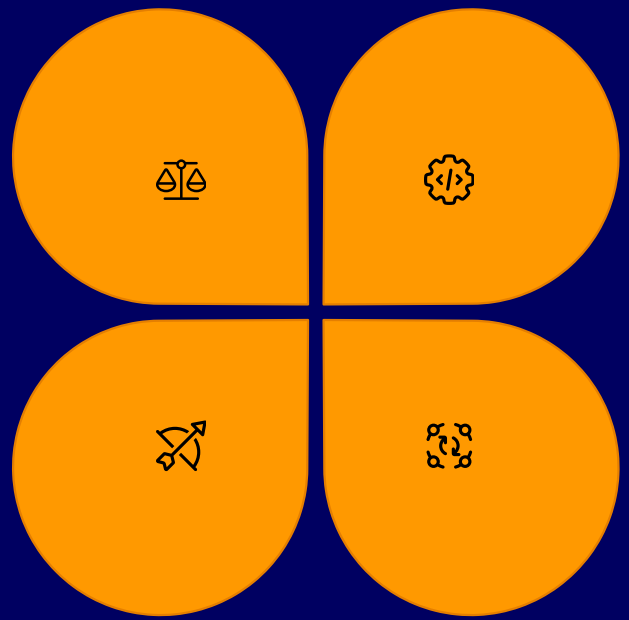
Desafios Comuns

Equilíbrio

Balancear segurança e experiência do usuário.

Ameaças

Proteger contra novos vetores de ataque.



Complexidade

Gerenciar múltiplos mecanismos de autenticação.

Integração

Conectar sistemas legados com soluções modernas.

Vantagens dos Tokens

Escalabilidade

Facilita crescimento de sistemas distribuídos.

Stateless

Elimina necessidade de armazenar estado no servidor.

Segurança

Base para autenticação moderna e segura.

Desafios dos Tokens



Proteção Adequada

Tokens devem ser armazenados e transmitidos com segurança.



Expiração Correta

Definir tempo de vida apropriado para cada contexto.



Renovação Segura

Implementar mecanismos robustos para renovar tokens expirados.

Implementação de Tokens em Python

```python
import jwt
import datetime

SECRET_KEY = 'minha_chave_secreta'

def criar_token(usuario):
    payload = {
        'user': usuario,
        'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
    }
    token = jwt.encode(payload, SECRET_KEY, algorithm='HS256')
    return token

def validar_token(token):
    try:
        decoded = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
        return decoded
    except jwt.ExpiredSignatureError:
        return "Token expirado"
    except jwt.InvalidTokenError:
        return "Token inválido"

# Exemplo de uso
token = criar_token("usuario_exemplo")
print("Token:", token)
print("Valida  o:", validar_token(token))
```

AWS STS (Security Token Service)

Credenciais Temporárias

Emite credenciais com validade limitada para acesso a recursos AWS.

Princípio do Menor Privilégio

Garante que usuários tenham apenas permissões necessárias.

Acesso Limitado

Credenciais válidas apenas pelo tempo necessário para a tarefa.



Vantagens do AWS STS

Segurança Aprimorada

Reduz risco de exposição de credenciais sensíveis.

Controle Granular

Permite definir exatamente quais recursos podem ser acessados.

Integração Segura

Facilita comunicação entre diferentes serviços AWS.



Implementação com AWS STS

```python
import boto3

sts_client = boto3.client('sts')

def assume_role(role_arn, session_name):
    response = sts_client.assume_role(
        RoleArn=role_arn,
        RoleSessionName=session_name,
        DurationSeconds=3600  # Credenciais válidas por 1 hora
    )
    return response['Credentials']

# Exemplo de uso
role_arn = "arn:aws:iam::123456789012:role/ExampleRole"
session_name = "ExemploSessao"
credentials = assume_role(role_arn, session_name)
print("Credenciais temporárias:", credentials)
```



Políticas de Acesso na AWS

O que são

Documentos JSON que definem permissões para usuários, grupos e roles.

Seguem a linguagem de políticas do IAM.

O que fazem?

Determinam quais ações são permitidas ou negadas em recursos específicos.

Permitem controle granular sobre acessos na AWS.

Elementos de uma Política

Effect

Define se a ação será permitida ou negada (Allow/Deny).

Action

Especifica quais operações podem ser executadas.

Resource

Indica em quais recursos AWS as ações se aplicam.

Condition

Estabelece condições adicionais para aplicação da política.



Estrutura de Políticas IAM

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "FirstStatement",
      "Effect": "Allow",
      "Action": ["iam:ChangePassword"],
      "Resource": "*"
    },
    {
      "Sid": "SecondStatement",
      "Effect": "Allow",
      "Action": "s3:ListAllMyBuckets",
      "Resource": "*"
    },
    {
      "Sid": "ThirdStatement",
      "Effect": "Allow",
      "Action": [
        "s3:List*",
        "s3:Get*"
      ],
      "Resource": [
        "arn:aws:s3:::amzn-s3-demo-bucket-confidential-data",
        "arn:aws:s3:::amzn-s3-demo-bucket-confidential-data/*"
      ],
      "Condition": {"Bool": {"aws:MultiFactorAuthPresent": "true"}}
    }
  ]
}
```

Tipos de Políticas IAM

Existem duas abordagens principais:

Baseadas em Recursos: As regras de acesso são definidas nos próprios recursos.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::123456789012:user/UsuarioExemplo"
            },
            "Action": [
                "s3:GetObject",
                "s3:PutObject"
            ],
            "Resource": "arn:aws:s3:::meu-bucket-exemplo/*"
        }
    ]
}
```

Diferente das **políticas baseadas em identidade**, essa política é anexada diretamente ao recurso (o bucket S3), e não ao usuário IAM.

Baseadas em Usuários: As permissões são concedidas diretamente aos usuários ou grupos.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "s3:ListAllMyBuckets",
            "Resource": "arn:aws:s3:::*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:GetObject",
                "s3:PutObject"
            ],
            "Resource": "arn:aws:s3:::meu-bucket-exemplo/*"
        }
    ]
}
```

Essa política pode ser anexada a um usuário, grupo ou função IAM para conceder as permissões apropriadas.

A escolha entre essas abordagens depende da necessidade de controle centralizado (baseado em usuários) ou descentralizado (baseado em recursos).

Exemplo: Função para validar a permissão de ação do usuário conforme regras.

```python
def verificar_permissao(usuario, acao):
    # Exemplo de regras predefinidas
    politicas = {
        "admin": ["criar", "ler", "atualizar", "deletar"],
        "usuario": ["ler"],
        "convidado": []
    }

    permissoes = politicas.get(usuario, [])
    if acao in permissoes:
        return True
    else:
        return False

# Exemplo de uso
usuario = "usuario"
acao = "atualizar"
if verificar_permissao(usuario, acao):
    print(f"O usuário {usuario} tem permiss o para {acao}.")
else:
    print(f"O usuário {usuario} N O tem permiss o para {acao}.")
```

Lembre-se: Políticas de Acesso

Gerenciamento Granular

Permite diferenciar permissões específicas entre recursos e identidades em seu ambiente AWS.

Proteção de Dados

Garante que apenas usuários autorizados realizem operações críticas em seus recursos na nuvem.

Configuração Cuidadosa

Requer entendimento profundo para evitar permissões excessivas ou lacunas de segurança.

A implementação eficaz de políticas de acesso é fundamental para manter a postura de segurança em ambientes AWS.



Amazon Cognito

Quando falamos de Autenticação e Autorização, não podemos deixar de fora o Amazon Cognito. Vamos relembrar o que já vimos desse serviço.

Gerenciamento de Usuários

Fornece autenticação e autorização para aplicativos web e móveis.

Integração Flexível

Conecta-se com provedores de identidade sociais e empresariais.

Experiência Otimizada

Simplifica login em aplicações multiplataforma.

Componentes do Amazon Cognito

User Pools

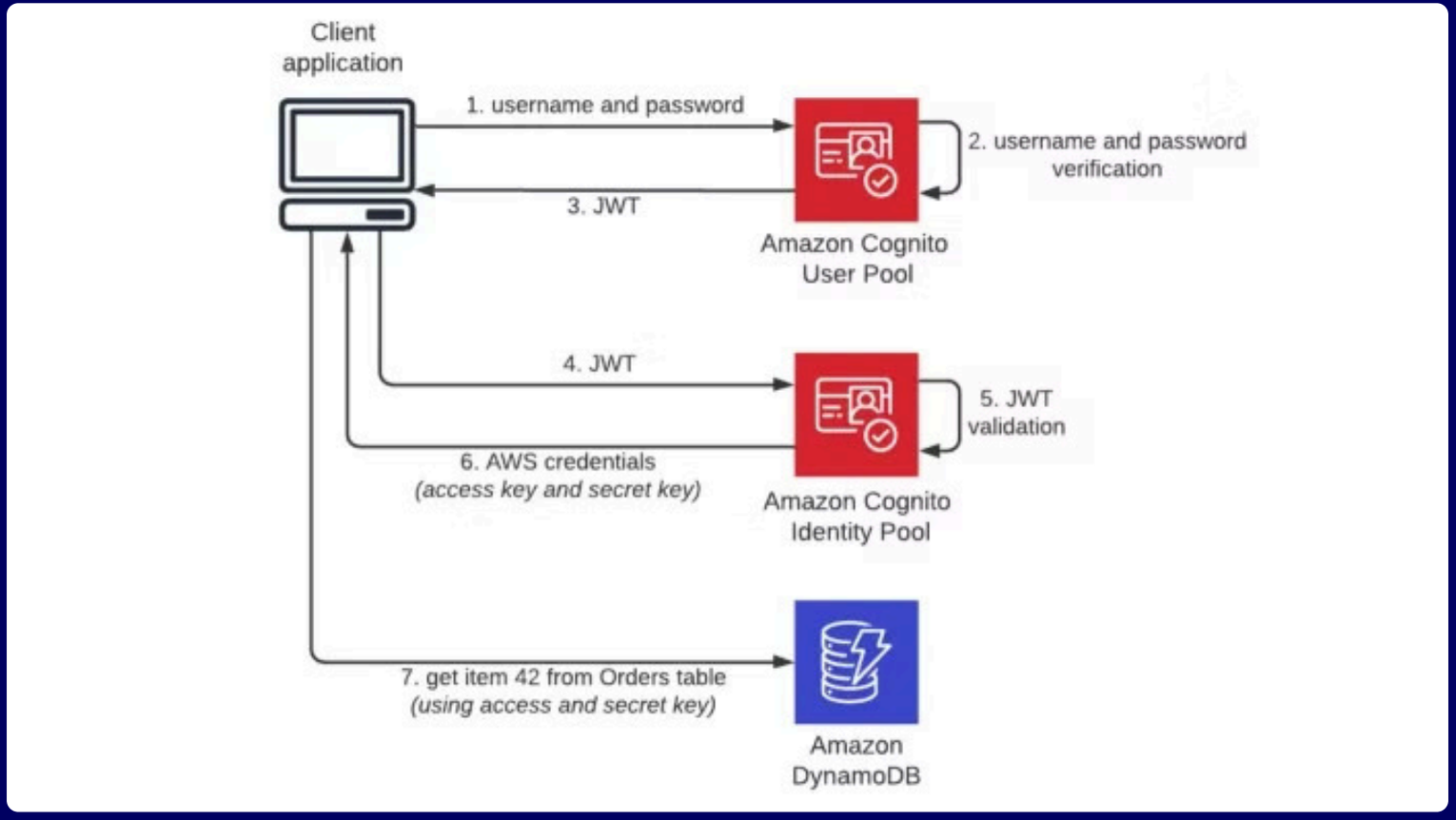
Diretórios de usuários que fornecem funcionalidades de cadastro e login.

Gerenciam perfis de usuário e atributos personalizados.

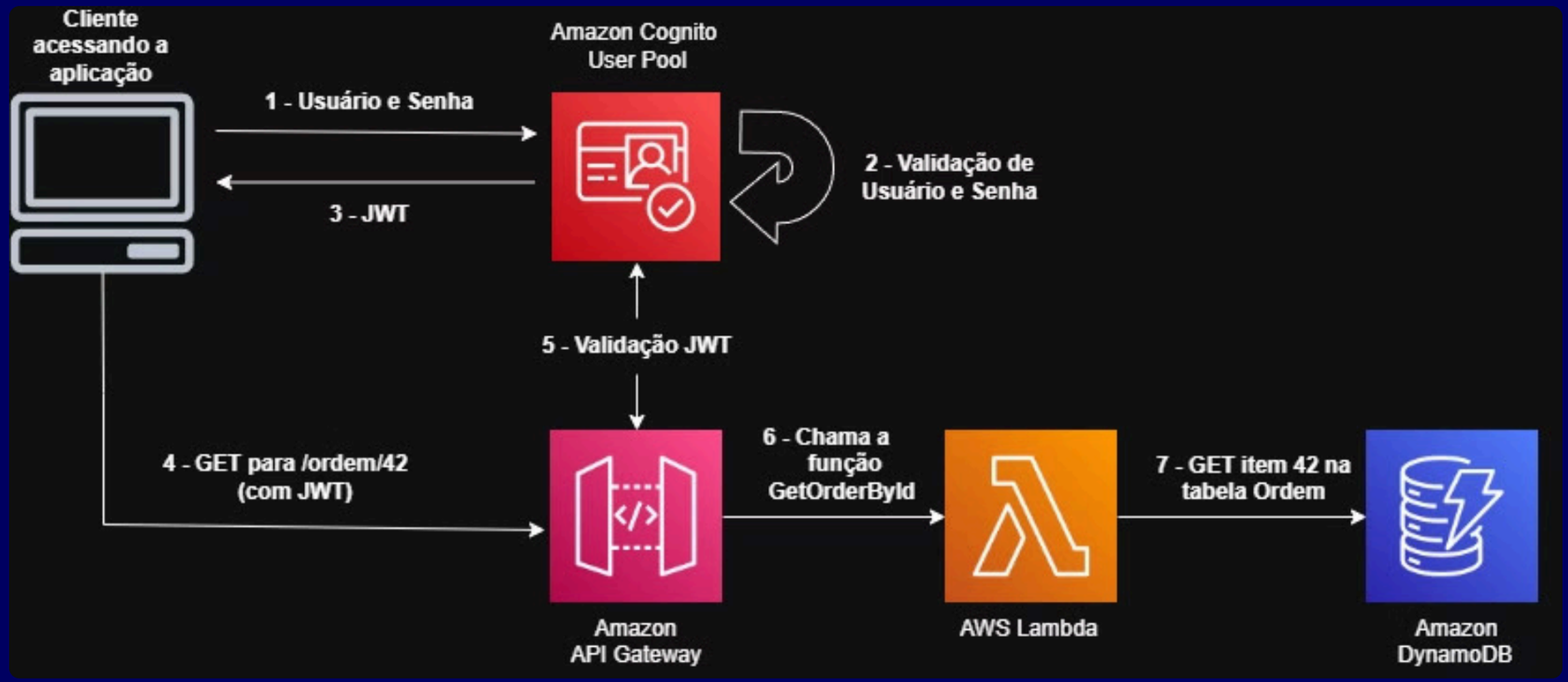
Identity Pools

Fornecem credenciais AWS temporárias para acesso a serviços.

Permitem acesso seguro a recursos AWS a partir de aplicações.



Fluxo de Autenticação no Cognito



Login do Usuário

Usuário fornece credenciais no aplicativo.

Validação

Cognito verifica as credenciais no User Pool.

Tokens

Após validação, Cognito emite tokens JWT.

Acesso Seguro

Aplicação usa tokens para acessar recursos protegidos.

Criação de Usuário em um Pool do Cognito

```python
import boto3

client = boto3.client('cognito-idp', region_name='us-east-1')
response = client.sign_up(
    ClientId='seu_client_id',
    Username='usuario_exemplo',
    Password='SenhaForte@123',
    UserAttributes=[
        {'Name': 'email', 'Value': 'usuario@exemplo.com'},
```



AWS IAM (Identity and Access Management)

Gerenciamento de Identidades

Controla quem pode acessar recursos AWS.

Controle de Permissões

Define o que cada identidade pode fazer.

Acesso Granular

Permite ajuste fino de permissões para cada recurso.



Componentes do IAM

Usuários (Users)

Identidades individuais com credenciais únicas e permanentes.

Grupos (Groups)

Conjuntos de usuários que compartilham permissões.

Funções (Roles)

Identidades assumidas por entidades para acesso temporário.

Políticas

Documentos que definem permissões específicas.

Melhores Práticas do IAM



Privilégio Mínimo

Conceda apenas permissões necessárias para cada função.



Uso de Grupos

Organize usuários em grupos para facilitar gerenciamento.



Credenciais Temporárias

Prefira roles e tokens temporários a chaves de acesso permanentes.

Um exemplo simples em Python para criar um usuário com permissões limitadas usando boto3

```python
import boto3

iam = boto3.client('iam')
response = iam.create_user(UserName='usuario_exemplo')
print("Usuário criado:", response)
```

Por que o IAM é fundamental para a Segurança na AWS

Segurança Zero-Trust

Implementa o princípio de "nunca confiar, sempre verificar"

Centralização de Identidades

Simplifica o gerenciamento em toda infraestrutura AWS

Governança e Conformidade

Facilita auditorias e demonstra conformidade regulatória

Automação de Segurança

Permite integração com CI/CD para segurança por código

A implementação correta do IAM é essencial para proteger recursos AWS contra acessos não autorizados e potenciais violações de dados.

Lembre-se: o IAM deve ser configurado seguindo o princípio de privilégio mínimo desde o início do projeto.

Por que o que vimos sobre Autenticação e Autorização é importante? (Visão Prática e da Certificação)

Visão Prática

- Protege dados sensíveis contra acessos não autorizados.
- Garante conformidade com regulamentações como LGPD e GDPR.
- Reduz superfície de ataque em ambientes cloud.
- Permite rastreabilidade e auditoria de ações.

Para a Certificação AWS

- Segurança representa até 26% das questões no exame.
- Exige compreensão profunda dos serviços IAM, Cognito, STS, etc.
- Avalia conhecimento de melhores práticas de implementação.